第5章 初始化与清理

随着计算机革命的发展，“不安全”的编程方式已逐渐成为编程代价高昂的主因之一。

*初始化*和*清理*（cleanup）正是涉及安全的两个问题。许多C程序的错误都源于程序员忘记初始化变量。特别是在使用程序库时，如果用户不知道如何初始化库的构件（或者是用户必须进行初始化的其他东西），更是如此。清理也是一个特殊问题，当使用完一个元素时，它对你也就不会有什么影响了，所以很容易把它忘记。这样一来，这个元素占用的资源就会一直得不到释放，结果是资源（尤其是内存）用尽。

C++引入了*构造器*（constructor）的概念，这是一个在创建对象时被自动调用的特殊方法。Java中也采用了构造器，并额外提供了“垃圾回收器”。对于不再使用的内存资源，垃圾回收器能自动将其释放。本章将讨论初始化和清理的相关问题，以及Java对它们提供的支持。

5.1 用构造器确保初始化

可以假想为编写的每个类都定义一个**initialize()**方法。该方法的名称提醒你在使用其对象之前，应首先调用**initialize()**。然而，这同时意味着用户必须记得自己去调用此方法。在Java中，通过提供构造器，类的设计者可确保每个对象都会得到初始化。创建对象时，如果其类具有构造器，Java就会在用户有能力操作对象之前自动调用相应的构造器，从而保证了初始化的进行。

接下来的问题就是如何命名这个方法。有两个问题：第一，所取的任何名字都可能与类的某个成员名称相冲突；第二，调用构造器是编译器的责任，所以必须让编译器知道应该调用哪个方法。C++语言中采用的解决方案看来最简单且更符合逻辑，所以在Java中也采用了这种方案：即构造器采用与类相同的名称。考虑到在初始化期间要自动调用构造器，这种做法就顺理成章了。

以下就是一个带有构造器的简单类：

```
//: initialization/SimpleConstructor.java
// Demonstration of a simple constructor.

class Rock {
  Rock() { // This is the constructor
    System.out.print("Rock ");
  }
}

public class SimpleConstructor {
  public static void main(String[] args) {
    for(int i = 0; i < 10; i++)
      new Rock();
  }
} /* Output:
Rock Rock Rock Rock Rock Rock Rock Rock Rock Rock
*///:~
```

现在，在创建对象时：

```
new Rock();
```

将会为对象分配存储空间，并调用相应的构造器。这就确保了在你能操作对象之前，它已经被

恰当地初始化了。

请注意，由于构造器的名称必须与类名完全相同，所以“每个方法首字母小写”的编码风格并不适用于构造器。

不接受任何参数的构造器叫做*默认构造器*，Java文档中通常使用术语*无参构造器*，但是默认构造器在Java出现之前已经使用许多年了，所以我仍旧倾向于使用它。但是和其他方法一样，构造器也能带有形式参数，以便指定如何创建对象。对上述例子稍加修改，即可使构造器接受一个参数：

156

```
//: initialization/SimpleConstructor2.java
// Constructors can have arguments.

class Rock2 {
  Rock2(int i) {
    System.out.print("Rock " + i + " ");
  }
}

public class SimpleConstructor2 {
  public static void main(String[] args) {
    for(int i = 0; i < 8; i++)
      new Rock2(i);
  }
} /* Output:
Rock 0 Rock 1 Rock 2 Rock 3 Rock 4 Rock 5 Rock 6 Rock 7
*///:~
```

有了构造器形式参数，就可以在初始化对象时提供实际参数。例如，假设类**Tree**有一个构造器，它接受一个整型变量来表示树的高度，就可以这样创建一个**Tree**对象：

```
Tree t = new Tree(12);  // 12-foot tree
```

如果**Tree(int)**是**Tree**类中唯一的构造器，那么编译器将不会允许你以其他任何方式创建**Tree**对象。

构造器有助于减少错误，并使代码更易于阅读。从概念上讲，“初始化”与“创建”是彼此独立的，然而在上面的代码中，你却找不到对**initialize()**方法的明确调用。在Java中，“初始化”和“创建”捆绑在一起，两者不能分离。

构造器是一种特殊类型的方法，因为它没有返回值。这与返回值为空（**void**）明显不同。对于空返回值，尽管方法本身不会自动返回什么，但仍可选择让它返回别的东西。构造器则不会返回任何东西，你别无选择（**new**表达式确实返回了对新建对象的引用，但构造器本身并没有任何返回值）。假如构造器具有返回值，并且允许人们自行选择返回类型，那么势必得让编译器知道该如何处理此返回值。

157

练习1：(1) 创建一个类，它包含一个未初始化的**String**引用。验证该引用被Java初始化成了**null**。

练习2：(2) 创建一个类，它包含一个在定义时就被初始化了的**String**域，以及另一个通过构造器初始化的**String**域。这两种方式有何差异？

5.2 方法重载

任何程序设计语言都具备的一项重要特性就是对名字的运用。当创建一个对象时，也就给此对象分配到的存储空间取了一个名字。所谓方法则是给某个动作取的名字。通过使用名字，你可以引用所有的对象和方法。名字起得好可以使系统更易于理解和修改。就好比写散文——目的是让读者易于理解。

将人类语言中存在细微差别的概念“映射”到程序设计语言中时，问题随之而生。在日常生活中，相同的词可以表达多种不同的含义——它们被“重载”了。特别是含义之间的差别很小时，这种方式十分有用。你可以说“清洗衬衫”、“清洗车”、“清洗狗”。但如果硬要这样说就显得很愚蠢：“以洗衬衫的方式洗衬衫”、“以洗车的方式洗车”、“以洗狗的方式洗狗”。这是因为听众根本不需要对所执行的动作做出明确的区分。大多数人类语言具有很强的“冗余”性，所以即使漏掉了几个词，仍然可以推断出含义。不需要对每个概念都使用不同的词汇——从具体的语境中就可以推断出含义。

大多数程序设计语言（尤其是C）要求为每个方法（在这些语言中经常称为函数）都提供一个独一无二的标识符。所以绝不能用名为**print()**的函数显示了整数之后，又用一个名为**print()**的函数显示浮点数——每个函数都要有唯一的名称。

在Java（和C++）里，构造器是强制重载方法名的另一个原因。既然构造器的名字已经由类名所决定，就只能有一个构造器名。那么要想用多种方式创建一个对象该怎么办呢？假设你要创建一个类，既可以用标准方式进行初始化，也可以从文件里读取信息来初始化。这就需要两个构造器：一个默认构造器，另一个取字符串作为形式参数——该字符串表示初始化对象所需的文件名称。由于都是构造器，所以它们必须有相同的名字，即类名。为了让方法名相同而形式参数不同的构造器同时存在，必须用到*方法重载*。同时，尽管方法重载是构造器所必需的，但它亦可应用于其他方法，且用法同样方便。

158

下面这个例子同时示范了重载的构造器和重载的方法：

```
//: initialization/Overloading.java
// Demonstration of both constructor
// and ordinary method overloading.
import static net.mindview.util.Print.*;

class Tree {
  int height;
  Tree() {
    print("Planting a seedling");
    height = 0;
  }
  Tree(int initialHeight) {
    height = initialHeight;
    print("Creating new Tree that is " +
      height + " feet tall");
  }
  void info() {
    print("Tree is " + height + " feet tall");
  }
  void info(String s) {
    print(s + ": Tree is " + height + " feet tall");
  }
}

public class Overloading {
  public static void main(String[] args) {
    for(int i = 0; i < 5; i++) {
      Tree t = new Tree(i);
      t.info();
      t.info("overloaded method");
    }
    // Overloaded constructor:
    new Tree();
  }
} /* Output:
Creating new Tree that is 0 feet tall
Tree is 0 feet tall
```

159

```
overloaded method: Tree is 0 feet tall
Creating new Tree that is 1 feet tall
Tree is 1 feet tall
overloaded method: Tree is 1 feet tall
Creating new Tree that is 2 feet tall
Tree is 2 feet tall
overloaded method: Tree is 2 feet tall
Creating new Tree that is 3 feet tall
Tree is 3 feet tall
overloaded method: Tree is 3 feet tall
Creating new Tree that is 4 feet tall
Tree is 4 feet tall
overloaded method: Tree is 4 feet tall
Planting a seedling
*///:~
```

创建**Tree**对象的时候，既可以不含参数，也可以用树的高度当参数。前者表示一棵树苗，后者表示已有一定高度的树木。要支持这种创建方式，得有一个默认构造器和一个采用现有高度作为参数的构造器。

或许你还想通过多种方式调用**info()**方法。例如，你想显示额外信息，可以用**info(String)**方法；没有的话就用**info()**。要是对明显相同的概念使用了不同的名字，那一定会让人很纳闷。好在有了方法重载，可以为两者使用相同的名字。

5.2.1 区分重载方法

要是几个方法有相同的名字，Java如何才能知道你指的是哪一个呢？其实规则很简单：每个重载的方法都必须有一个独一无二的参数类型列表。

稍加思考，就会觉得这是合理的。毕竟，对于名字相同的方法，除了参数类型的差异以外，还有什么办法能把它们区别开呢？

甚至参数顺序的不同也足以区分两个方法。不过，一般情况下别这么做，因为这会使代码难以维护：

```
//: initialization/OverloadingOrder.java
// Overloading based on the order of the arguments.
import static net.mindview.util.Print.*;
public class OverloadingOrder {
  static void f(String s, int i) {
    print("String: " + s + ", int: " + i);
  }
  static void f(int i, String s) {
    print("int: " + i + ", String: " + s);
  }
  public static void main(String[] args) {
    f("String first", 11);
    f(99, "Int first");
  }
} /* Output:
String: String first, int: 11
int: 99, String: Int first
*///:~
```

160

上例中两个**print()**方法虽然声明了相同的参数，但顺序不同，因此得以区分。

5.2.2 涉及基本类型的重载

基本类型能从一个“较小”的类型自动提升至一个“较大”的类型，此过程一旦牵涉到重载，可能会造成一些混淆。以下例子说明了将基本类型传递给重载方法时发生的情况：

```
//: initialization/PrimitiveOverloading.java
// Promotion of primitives and overloading.
import static net.mindview.util.Print.*;
```

```
public class PrimitiveOverloading {
  void f1(char x) { printnb("f1(char) "); }
  void f1(byte x) { printnb("f1(byte) "); }
  void f1(short x) { printnb("f1(short) "); }
  void f1(int x) { printnb("f1(int) "); }
  void f1(long x) { printnb("f1(long) "); }
  void f1(float x) { printnb("f1(float) "); }
  void f1(double x) { printnb("f1(double) "); }

  void f2(byte x) { printnb("f2(byte) "); }
  void f2(short x) { printnb("f2(short) "); }
  void f2(int x) { printnb("f2(int) "); }
  void f2(long x) { printnb("f2(long) "); }
  void f2(float x) { printnb("f2(float) "); }
  void f2(double x) { printnb("f2(double) "); }

  void f3(short x) { printnb("f3(short) "); }
  void f3(int x) { printnb("f3(int) "); }
  void f3(long x) { printnb("f3(long) "); }
  void f3(float x) { printnb("f3(float) "); }
  void f3(double x) { printnb("f3(double) "); }

  void f4(int x) { printnb("f4(int) "); }
  void f4(long x) { printnb("f4(long) "); }
  void f4(float x) { printnb("f4(float) "); }
  void f4(double x) { printnb("f4(double) "); }

  void f5(long x) { printnb("f5(long) "); }
  void f5(float x) { printnb("f5(float) "); }
  void f5(double x) { printnb("f5(double) "); }

  void f6(float x) { printnb("f6(float) "); }
  void f6(double x) { printnb("f6(double) "); }

  void f7(double x) { printnb("f7(double) "); }

  void testConstVal() {
    printnb("5: ");
    f1(5);f2(5);f3(5);f4(5);f5(5);f6(5);f7(5); print();
  }
  void testChar() {
    char x = 'x';
    printnb("char: ");
    f1(x);f2(x);f3(x);f4(x);f5(x);f6(x);f7(x); print();
  }
  void testByte() {
    byte x = 0;
    printnb("byte: ");
    f1(x);f2(x);f3(x);f4(x);f5(x);f6(x);f7(x); print();
  }
  void testShort() {
    short x = 0;
    printnb("short: ");
    f1(x);f2(x);f3(x);f4(x);f5(x);f6(x);f7(x); print();
  }
  void testInt() {
    int x = 0;
    printnb("int: ");
    f1(x);f2(x);f3(x);f4(x);f5(x);f6(x);f7(x); print();
  }
  void testLong() {
    long x = 0;
    printnb("long: ");
    f1(x);f2(x);f3(x);f4(x);f5(x);f6(x);f7(x); print();
  }
  void testFloat() {
    float x = 0;
```

161

162

```
    printnb("float: ");
    f1(x);f2(x);f3(x);f4(x);f5(x);f6(x);f7(x); print();
  }
  void testDouble() {
    double x = 0;
    printnb("double: ");
    f1(x);f2(x);f3(x);f4(x);f5(x);f6(x);f7(x); print();
  }
  public static void main(String[] args) {
    PrimitiveOverloading p =
      new PrimitiveOverloading();
    p.testConstVal();
    p.testChar();
    p.testByte();
    p.testShort();
    p.testInt();
    p.testLong();
    p.testFloat();
    p.testDouble();
  }
} /* Output:
5: f1(int) f2(int) f3(int) f4(int) f5(long) f6(float)
f7(double)
char: f1(char) f2(int) f3(int) f4(int) f5(long) f6(float)
f7(double)
byte: f1(byte) f2(byte) f3(short) f4(int) f5(long)
f6(float) f7(double)
short: f1(short) f2(short) f3(short) f4(int) f5(long)
f6(float) f7(double)
int: f1(int) f2(int) f3(int) f4(int) f5(long) f6(float)
f7(double)
long: f1(long) f2(long) f3(long) f4(long) f5(long)
f6(float) f7(double)
float: f1(float) f2(float) f3(float) f4(float) f5(float)
f6(float) f7(double)
double: f1(double) f2(double) f3(double) f4(double)
f5(double) f6(double) f7(double)
*///:~
```

你会发现常数值5被当作**int**值处理，所以如果有某个重载方法接受**int**型参数，它就会被调用。至于其他情况，如果传入的数据类型（实际参数类型）小于方法中声明的形式参数类型，实际数据类型就会被提升。**char**型略有不同，如果无法找到恰好接受**char**参数的方法，就会把**char**直接提升至**int**型。

如果传入的实际参数大于重载方法声明的形式参数，会出现什么情况呢？修改上述程序，就能得到答案：

```
//: initialization/Demotion.java
// Demotion of primitives and overloading.
import static net.mindview.util.Print.*;

public class Demotion {
  void f1(char x) { print("f1(char)"); }
  void f1(byte x) { print("f1(byte)"); }
  void f1(short x) { print("f1(short)"); }
  void f1(int x) { print("f1(int)"); }
  void f1(long x) { print("f1(long)"); }
  void f1(float x) { print("f1(float)"); }
  void f1(double x) { print("f1(double)"); }

  void f2(char x) { print("f2(char)"); }
  void f2(byte x) { print("f2(byte)"); }
  void f2(short x) { print("f2(short)"); }
  void f2(int x) { print("f2(int)"); }
  void f2(long x) { print("f2(long)"); }
```

```
    void f2(float x) { print("f2(float)"); }

    void f3(char x) { print("f3(char)"); }
    void f3(byte x) { print("f3(byte)"); }
    void f3(short x) { print("f3(short)"); }
    void f3(int x) { print("f3(int)"); }
    void f3(long x) { print("f3(long)"); }

    void f4(char x) { print("f4(char)"); }
    void f4(byte x) { print("f4(byte)"); }
    void f4(short x) { print("f4(short)"); }
    void f4(int x) { print("f4(int)"); }
    void f5(char x) { print("f5(char)"); }
    void f5(byte x) { print("f5(byte)"); }
    void f5(short x) { print("f5(short)"); }

    void f6(char x) { print("f6(char)"); }
    void f6(byte x) { print("f6(byte)"); }

    void f7(char x) { print("f7(char)"); }

    void testDouble() {
      double x = 0;
      print("double argument:");
      f1(x);f2((float)x);f3((long)x);f4((int)x);
      f5((short)x);f6((byte)x);f7((char)x);
    }
    public static void main(String[] args) {
      Demotion p = new Demotion();
      p.testDouble();
    }
  } /* Output:
  double argument:
  f1(double)
  f2(float)
  f3(long)
  f4(int)
  f5(short)
  f6(byte)
  f7(char)
  *///:~
```

164

在这里，方法接受较小的基本类型作为参数。如果传入的实际参数较大，就得通过类型转换来执行窄化转换。如果不这样做，编译器就会报错。

5.2.3 以返回值区分重载方法

读者可能会想："在区分重载方法的时候，为什么只能以类名和方法的形参列表作为标准呢？能否考虑用方法的返回值来区分呢？"比如下面两个方法，虽然它们有同样的名字和形式参数，但却很容易区分它们：

```
void f() {}
int f() { return 1; }
```

165

只要编译器可以根据语境明确判断出语义，比如在**int x=f()**中，那么的确可以据此区分重载方法。不过，有时你并不关心方法的返回值，你想要的是方法调用的其他效果（这常被称为"为了副作用而调用"），这时你可能会调用方法而忽略其返回值。所以，如果像下面这样调用方法：

```
f();
```

此时Java如何才能判断该调用哪一个**f()**呢？别人该如何理解这种代码呢？因此，根据方法的返回值来区分重载方法是行不通的。

5.3 默认构造器

如前所述，默认构造器（又名“无参”构造器）是没有形式参数的——它的作用是创建一个“默认对象”。如果你写的类中没有构造器，则编译器会自动帮你创建一个默认构造器。例如：

```
//: initialization/DefaultConstructor.java

class Bird {}

public class DefaultConstructor {
  public static void main(String[] args) {
    Bird b = new Bird(); // Default!
  }
} ///:~
```

表达式

```
new Bird()
```

行创建了一个新对象，并调用其默认构造器——即使你没有明确定义它。没有它的话，就没有方法可调用，就无法创建对象。但是，如果已经定义了一个构造器（无论是否有参数），编译器就不会帮你自动创建默认构造器：

```
//: initialization/NoSynthesis.java

class Bird2 {
  Bird2(int i) {}
  Bird2(double d) {}
}

public class NoSynthesis {
  public static void main(String[] args) {
    //! Bird2 b = new Bird2(); // No default
    Bird2 b2 = new Bird2(1);
    Bird2 b3 = new Bird2(1.0);
  }
} ///:~
```

166

要是你这样写：

```
new Bird2()
```

编译器就会报错：没有找到匹配的构造器。这就好比，要是你没有提供任何构造器，编译器会认为“你需要一个构造器，让我给你制造一个吧”，但假如你已写了一个构造器，编译器则会认为“啊，你已写了一个构造器，所以你知道你在做什么；你是刻意省略了默认构造器。”

练习3：(1) 创建一个带默认构造器（即无参构造器）的类，在构造器中打印一条消息。为这个类创建一个对象。

练习4：(1) 为前一个练习中的类添加一个重载构造器，令其接受一个字符串参数，并在构造器中把你自己的消息和接收的参数一起打印出来。

练习5：(2) 创建一个名为**Dog**的类，它具有重载的**bark()**方法。此方法应根据不同的基本数据类型进行重载，并根据被调用的版本，打印出不同类型的狗吠（barking）、咆哮（howling）等信息。编写**main()**来调用所有不同版本的方法。

练习6：(1) 修改前一个练习的程序，让两个重载方法各自接受两个类型的不同的参数，但二者顺序相反。验证其是否工作。

练习7：(1) 创建一个没有构造器的类，并在**main()**中创建其对象，用以验证编译器是否真的自动加入了默认构造器。

5.4 this关键字

如果有同一类型的两个对象，分别是a和b。你可能想知道，如何才能让这两个对象都能调用peel()方法呢：

167

```
//: initialization/BananaPeel.java

class Banana { void peel(int i) { /* ... */ } }

public class BananaPeel {
  public static void main(String[] args) {
    Banana a = new Banana(),
           b = new Banana();
    a.peel(1);
    b.peel(2);
  }
} ///:~
```

如果只有一个peel()方法，它如何知道是被a还是被b所调用的呢？

为了能用简便、面向对象的语法来编写代码——即“发送消息给对象”，编译器做了一些幕后工作。它暗自把“所操作对象的引用”作为第一个参数传递给peel()。所以上述两个方法的调用就变成了这样：

```
Banana.peel(a, 1);
Banana.peel(b, 2);
```

这是内部的表示形式。我们并不能这样书写代码，并试图通过编译；但这种写法的确能帮你了解实际所发生的事情。

假设你希望在方法的内部获得对当前对象的引用。由于这个引用是由编译器“偷偷”传入的，所以没有标识符可用。但是，为此有个专门的关键字：this。this关键字只能在方法内部使用，表示对“调用方法的那个对象”的引用。this的用法和其他对象引用并无不同。但要注意，如果在方法内部调用同一个类的另一个方法，就不必使用this，直接调用即可。当前方法中的this引用会自动应用于同一类中的其他方法。所以可以这样写代码：

```
//: initialization/Apricot.java
public class Apricot {
  void pick() { /* ... */ }
  void pit() { pick(); /* ... */ }
} ///:~
```

168

在pit()内部，你可以写this.pick()，但无此必要[⊖]。编译器能帮你自动添加。只有当需要明确指出对当前对象的引用时，才需要使用this关键字。例如，当需要返回对当前对象的引用时，就常常在return语句里这样写：

```
//: initialization/Leaf.java
// Simple use of the "this" keyword.

public class Leaf {
  int i = 0;
  Leaf increment() {
    i++;
    return this;
  }
```

⊖ 有些人执意将this放在每一个方法调用和字段引用前，认为这样“更清楚更明确”。但是，千万别这么做。我们使用高级语言的原因之一就是它们能帮我们做一些事情。要是你把this放在一些没必要的地方，就会使读你程序的人不知所措，因为别人写的代码不会到处使用this。人们期望只在必要处使用this。遵循一种一致而直观的编程风格能节省时间和金钱。

```
  void print() {
    System.out.println("i = " + i);
  }
  public static void main(String[] args) {
    Leaf x = new Leaf();
    x.increment().increment().increment().print();
  }
} /* Output:
i = 3
*///:~
```

由于**increment()**通过**this**关键字返回了对当前对象的引用，所以很容易在一条语句里对同一个对象执行多次操作。

this关键字对于将当前对象传递给其他方法也很有用：

```
//: initialization/PassingThis.java

class Person {
  public void eat(Apple apple) {
    Apple peeled = apple.getPeeled();
    System.out.println("Yummy");
  }
}

class Peeler {
  static Apple peel(Apple apple) {
    // ... remove peel
    return apple; // Peeled
  }
}

class Apple {
  Apple getPeeled() { return Peeler.peel(this); }
}

public class PassingThis {
  public static void main(String[] args) {
    new Person().eat(new Apple());
  }
} /* Output:
Yummy
*///:~
```

169

Apple需要调用**Peeler.peel()**方法，它是一个外部的工具方法，将执行由于某种原因而必须放在**Apple**外部的操作（也许是因为该外部方法要应用于许多不同的类，而你却不想重复这些代码）。为了将其自身传递给外部方法，**Apple**必须使用**this**关键字。

练习8：(1) 编写具有两个方法的类，在第一个方法内调用第二个方法两次：第一次调用时不使用**this**关键字，第二次调用时使用**this**关键字——这里只是为了验证它是起作用的，你不应该在实践中使用这种方式。

5.4.1 在构造器中调用构造器

可能为一个类写了多个构造器，有时可能想在一个构造器中调用另一个构造器，以避免重复代码。可用**this**关键字做到这一点。

通常写**this**的时候，都是指“这个对象”或者“当前对象”，而且它本身表示对当前对象的引用。在构造器中，如果为**this**添加了参数列表，那么就有了不同的含义。这将产生对符合此参数列表的某个构造器的明确调用；这样，调用其他构造器就有了直接的途径：

170

```
//: initialization/Flower.java
// Calling constructors with "this"
import static net.mindview.util.Print.*;
```

```
public class Flower {
  int petalCount = 0;
  String s = "initial value";
  Flower(int petals) {
    petalCount = petals;
    print("Constructor w/ int arg only, petalCount= "
      + petalCount);
  }
  Flower(String ss) {
    print("Constructor w/ String arg only, s = " + ss);
    s = ss;
  }
  Flower(String s, int petals) {
    this(petals);
//!    this(s); // Can't call two!
    this.s = s; // Another use of "this"
    print("String & int args");
  }
  Flower() {
    this("hi", 47);
    print("default constructor (no args)");
  }
  void printPetalCount() {
//! this(11); // Not inside non-constructor!
    print("petalCount = " + petalCount + " s = "+ s);
  }
  public static void main(String[] args) {
    Flower x = new Flower();
    x.printPetalCount();
  }
} /* Output:
Constructor w/ int arg only, petalCount= 47
String & int args
default constructor (no args)
petalCount = 47 s = hi
*///:~
```

171

构造器**Flower(String s,int petals)**表明：尽管可以用**this**调用一个构造器，但却不能调用两个。此外，必须将构造器调用置于最起始处，否则编译器会报错。

这个例子也展示了**this**的另一种用法。由于参数**s**的名称和数据成员**s**的名字相同，所以会产生歧义。使用**this.s**来代表数据成员就能解决这个问题。在Java程序代码中经常出现这种写法，本书中也常这么写。

printPetalCount()方法表明，除构造器之外，编译器禁止在其他任何方法中调用构造器。

练习9：(1)编写具有两个（重载）构造器的类，并在第一个构造器中通过**this**调用第二个构造器。

5.4.2 static的含义

了解**this**关键字之后，就能更全面地理解**static**（静态）方法的含义。**static**方法就是没有**this**的方法。在**static**方法的内部不能调用非静态方法[⊖]，反过来倒是可以的。而且可以在没有创建任何对象的前提下，仅仅通过类本身来调用**static**方法。这实际上正是**static**方法的主要用途。它很像全局方法。Java中禁止使用全局方法，但你在类中置入**static**方法就可以访问其他**static**方法和**static**域。

有些人认为**static**方法不是“面向对象”的，因为它们的确具有全局函数的语义；使用**static**

⊖ 这不是完全不可能。如果你传递一个对象的引用到静态方法里（静态方法可以创建其自身的对象），然后通过这个引用（和**this**效果相同），你就可以调用非静态方法和访问非静态数据成员了。但通常要达到这样的效果，你只需写一个非静态方法即可。

方法时，由于不存在**this**，所以不是通过“向对象发送消息”的方式来完成的。的确，要是在代码中出现了大量的**static**方法，就该重新考虑自己的设计了。然而，**static**的概念有其实用之处，许多时候都要用到它。至于它是否真的“面向对象”，就留给理论家去讨论吧。事实上，Smalltalk语言里的“类方法”就是与**static**方法相对应的。 172

5.5 清理：终结处理和垃圾回收

程序员都了解初始化的重要性，但常常会忘记同样也重要的清理工作。毕竟，谁需要清理一个int呢？但在使用程序库时，把一个对象用完后就“弃之不顾”的做法并非总是安全的。当然，Java有垃圾回收器负责回收无用对象占据的内存资源。但也有特殊情况：假定你的对象（并非使用new）获得了一块“特殊”的内存区域，由于垃圾回收器只知道释放那些经由new分配的内存，所以它不知道该如何释放该对象的这块“特殊”内存。为了应对这种情况，Java允许在类中定义一个名为**finalize()**的方法。它的工作原理“假定”是这样的：一旦垃圾回收器准备好释放对象占用的存储空间，将首先调用其**finalize()**方法，并且在下一次垃圾回收动作发生时，才会真正回收对象占用的内存。所以要是你打算用**finalize()**，就能在垃圾回收时刻做一些重要的清理工作。

这里有一个潜在的编程陷阱，因为有些程序员（特别是C++程序员）刚开始可能会误把**finalize()**当作C++中的*析构函数*（C++中销毁对象必须用到这个函数）。所以有必要明确区分一下：在C++中，对象一定会被销毁（如果程序中没有缺陷的话）；而Java里的对象却并非总是被垃圾回收。或者换句话说：

1. *对象可能不被垃圾回收。*
2. *垃圾回收并不等于“析构”。*

牢记这些，就能远离困扰。这意味着在你不再需要某个对象之前，如果必须执行某些动作，那么你得自己去做。Java并未提供“析构函数”或相似的概念，要做类似的清理工作，必须自己动手创建一个执行清理工作的普通方法。例如，假设某个对象在创建过程中会将自己绘制到屏幕上，如果不是明确地从屏幕上将其擦除，它可能永远得不到清理。如果在**finalize()**里加入某种擦除功能，当“垃圾回收”发生时（不能保证一定会发生），**finalize()**得到了调用，图像就会被擦除。要是“垃圾回收”没有发生，图像就会一直保留下来。 173

也许你会发现，只要程序没有濒临存储空间用完的那一刻，对象占用的空间就总也得不到释放。如果程序执行结束，并且垃圾回收器一直都没有释放你创建的任何对象的存储空间，则随着程序的退出，那些资源也会全部交还给操作系统。这个策略是恰当的，因为垃圾回收本身也有开销，要是不使用它，那就不用支付这部分开销了。

5.5.1 finalize()的用途何在

此时，读者已经明白了不该将**finalize()**作为通用的清理方法。那么，**finalize()**的真正用途是什么呢？

这引出了要记住的第三点：

3. *垃圾回收只与内存有关。*

也就是说，使用垃圾回收器的唯一原因是为了回收程序不再使用的内存。所以对于与垃圾回收有关的任何行为来说（尤其是**finalize()**方法），它们也必须同内存及其回收有关。

但这是否意味着要是对象中含有其他对象，**finalize()**就应该明确释放那些对象呢？不，无论对象是如何创建的，垃圾回收器都会负责释放对象占据的所有内存。这就将对**finalize()**的需求限制到一种特殊情况，即通过某种创建对象方式以外的方式为对象分配了存储空间。不过，

读者也看到了，Java中一切皆为对象，那这种特殊情况是怎么回事呢？

看来之所以要有**finalize()**，是由于在分配内存时可能采用了类似C语言中的做法，而非Java中的通常做法。这种情况主要发生在使用“本地方法”的情况下，本地方法是一种在Java中调用非Java代码的方式（关于本地方法的讨论见本书电子版第2版，在www.MindView.net网站上有收录）。本地方法目前只支持C和C++，但它们可以调用其他语言写的代码，所以实际上可以调用任何代码。在非Java代码中，也许会调用C的**malloc()**函数系列来分配存储空间，而且除非调用了**free()**函数，否则存储空间将得不到释放，从而造成内存泄露。当然，**free()**是C和C++中的函数，所以需要在**finalize()**中用本地方法调用它。

174

至此，读者或许已经明白了不要过多地使用**finalize()**的道理了[⊖]。对，它确实不是进行普通的清理工作的合适场所。那么，普通的清理工作应该在哪里执行呢？

5.5.2 你必须实施清理

要清理一个对象，用户必须在需要清理的时刻调用执行清理动作的方法。这听起来似乎很简单，但却与C++中的“析构函数”的概念稍有抵触。在C++中，所有对象都会被销毁，或者说，应该被销毁。如果在C++中创建了一个局部对象（也就是在堆栈上创建，这在Java中行不通），此时的销毁动作发生在以“右花括号”为边界的、此对象作用域的末尾处。如果对象是用**new**创建的（类似于Java中），那么当程序员调用C++的delete操作符时（Java没有这个命令），就会调用相应的析构函数。如果程序员忘记调用**delete**，那么永远不会调用析构函数，这样就会出现内存泄露，对象的其他部分也不会得到清理。这种缺陷很难跟踪，这也是让C++程序员转向Java的一个主要因素。

相反，Java不允许创建局部对象，必须使用**new**创建对象。在Java中，也没有用于释放对象的**delete**，因为垃圾回收器会帮助你释放存储空间。甚至可以肤浅地认为，正是由于垃圾收集机制的存在，使得Java没有析构函数。然而，随着学习的深入，读者就会明白垃圾回收器的存在并不能完全代替析构函数。（而且绝对不能直接调用**finalize()**，所以这也不是一种解决方案。）如果希望进行除释放存储空间之外的清理工作，还是得明确调用某个恰当的Java方法。这就等同于使用析构函数了，只是没有它方便。

记住，无论是“垃圾回收”还是“终结”，都不保证一定会发生。如果Java虚拟机（JVM）并未面临内存耗尽的情形，它是不会浪费时间去执行垃圾回收以恢复内存的。

175

5.5.3 终结条件

通常，不能指望**finalize()**，必须创建其他的“清理”方法，并且明确地调用它们。看来，**finalize()**只能存在于程序员很难用到的一些晦涩用法里了。不过，**finalize()**还有一个有趣的用法，它并不依赖于每次都要对**finalize()**进行调用，这就是对象终结条件[⊖]的验证。

当对某个对象不再感兴趣——也就是它可以被清理了，这个对象应该处于某种状态，使它占用的内存可以被安全地释放。例如，要是对象代表了一个打开的文件，在对象被回收前程序员应该关闭这个文件。只要对象中存在没有被适当清理的部分，程序就存在很隐晦的缺陷。**finalize()**可以用来最终发现这种情况——尽管它并不总是会被调用。如果某次**finalize()**的动作使得缺陷被发现，那么就可据此找出问题所在——这才是人们真正关心的。

以下是个简单的例子，示范了**finalize()**可能的使用方式：

⊖ Joshua Bloch在题为“避免使用终结函数”一节中走的更远，他提到：“终结函数无法预料，常常是危险的，总之是多余的。”《Effective Java》，第20页，(Addison-Wesley 2001)。

⊖ 这个术语是在由Bill Venners (www.artima.com)和我一同开的培训班上发明的。

```
//: initialization/TerminationCondition.java
// Using finalize() to detect an object that
// hasn't been properly cleaned up.

class Book {
  boolean checkedOut = false;
  Book(boolean checkOut) {
    checkedOut = checkOut;
  }
  void checkIn() {
    checkedOut = false;
  }
  protected void finalize() {
    if(checkedOut)
      System.out.println("Error: checked out");
    // Normally, you'll also do this:
    // super.finalize(); // Call the base-class version
  }
}

public class TerminationCondition {
  public static void main(String[] args) {
    Book novel = new Book(true);
    // Proper cleanup:
    novel.checkIn();
    // Drop the reference, forget to clean up:
    new Book(true);
    // Force garbage collection & finalization:
    System.gc();
  }
} /* Output:
Error: checked out
*///:~
```

176

本例的终结条件是：所有的**Book**对象在被当作垃圾回收前都应该被签入（check in）。但在**main()**方法中，由于程序员的错误，有一本书未被签入。要是没有**finalize()**来验证终结条件，将很难发现这种缺陷。

注意，**System.gc()**用于强制进行终结动作。即使不这么做，通过重复地执行程序（假设程序将分配大量的存储空间而导致垃圾回收动作的执行），最终也能找出错误的**Book**对象。

你应该总是假设基类版本的**finalize()**也要做某些重要的事情，因此要使用super来调用它，就像在**Book.finalize()**中看到的那样。在本例中，它被注释掉了，因为它需要进行异常处理，而我们还没有介绍过这部分内容。

练习10：(2) 编写具有**finalize()**方法的类，并在方法中打印消息。在**main()**中为该类创建一个对象。试解释这个程序的行为。

练习11：(4) 修改前一个练习的程序，让你的**finalize()**总会被调用。

练习12：(4) 编写名为**Tank**的类，此类的状态可以是“满的”或“空的”。其终结条件是：对象被清理时必须处于空状态。请编写**finalize()**以检验终结条件是否成立。在**main()**中测试**Tank** 可能发生的几种使用方式。

177

5.5.4 垃圾回收器如何工作

在以前所用过的程序语言中，在堆上分配对象的代价十分高昂，因此读者自然会觉得Java中所有对象（基本类型除外）都在堆上分配的方式也非常高昂。然而，垃圾回收器对于提高对象的创建速度，却具有明显的效果。听起来很奇怪——存储空间的释放竟然会影响存储空间的分配，但这确实是某些Java虚拟机的工作方式。这也意味着，Java从堆分配空间的速度，可以和其他语言从堆栈上分配空间的速度相媲美。

打个比方，你可以把C++里的堆想像成一个院子，里面每个对象都负责管理自己的地盘。一段时间以后，对象可能被销毁，但地盘必须加以重用。在某些Java虚拟机中，堆的实现截然不同：它更像一个传送带，每分配一个新对象，它就往前移动一格。这意味着对象存储空间的分配速度非常快。Java的“堆指针”只是简单地移动到尚未分配的区域，其效率比得上C++在堆栈上分配空间的效率。当然，实际过程中在簿记工作方面还有少量额外开销，但比不上查找可用空间开销大。

读者也许已经意识到了，Java中的堆未必完全像传送带那样工作。要真是那样的话，势必会导致频繁的内存页面调度——将其移进移出硬盘，因此会显得需要拥有比实际需要更多的内存。页面调度会显著地影响性能，最终，在创建了足够多的对象之后，内存资源将耗尽。其中的秘密在于垃圾回收器的介入。当它工作时，将一面回收空间，一面使堆中的对象紧凑排列，这样“堆指针”就可以很容易移动到更靠近传送带的开始处，也就尽量避免了页面错误。通过垃圾回收器对对象重新排列，实现了一种高速的、有无限空间可供分配的堆模型。

要想更好地理解Java中的垃圾回收，先了解其他系统中的垃圾回收机制将会很有帮助。*引用记数*是一种简单但速度很慢的垃圾回收技术。每个对象都含有一个引用记数器，当有引用连接至对象时，引用计数加1。当引用离开作用域或被置为**null**时，引用计数减1。虽然管理引用记数的开销不大，但这项开销在整个程序生命周期中将持续发生。垃圾回收器会在含有全部对象的列表上遍历，当发现某个对象的引用计数为0时，就释放其占用的空间（但是，引用记数模式经常会在记数值变为0时立即释放对象）。这种方法有个缺陷，如果对象之间存在循环引用，可能会出现“对象应该被回收，但引用计数却不为零”的情况。对垃圾回收器而言，定位这样的交互自引用的对象组所需的工作量极大。引用记数常用来说明垃圾收集的工作方式，但似乎从未被应用于任何一种Java虚拟机实现中。

178

在一些更快的模式中，垃圾回收器并非基于引用记数技术。它们依据的思想是：对任何“活”的对象，一定能最终追溯到其存活在堆栈或静态存储区之中的引用。这个引用链条可能会穿过数个对象层次。由此，如果从堆栈和静态存储区开始，遍历所有的引用，就能找到所有“活”的对象。对于发现的每个引用，必须追踪它所引用的对象，然后是此对象包含的所有引用，如此反复进行，直到“根源于堆栈和静态存储区的引用”所形成的网络全部被访问为止。你所访问过的对象必须都是“活”的。注意，这就解决了“交互自引用的对象组”的问题——这种现象根本不会被发现，因此也就被自动回收了。

在这种方式下，Java虚拟机将采用一种*自适应*的垃圾回收技术。至于如何处理找到的存活对象，取决于不同的Java虚拟机实现。有一种做法名为*停止-复制*（stop-and-copy）。显然这意味着，先暂停程序的运行（所以它不属于后台回收模式），然后将所有存活的对象从当前堆复制到另一个堆，没有被复制的全部都是垃圾。当对象被复制到新堆时，它们是一个挨着一个的，所以新堆保持紧凑排列，然后就可以按前述方法简单、直接地分配新空间了。

当把对象从一处搬到另一处时，所有指向它的那些引用都必须修正。位于堆或静态存储区的引用可以直接被修正，但可能还有其他指向这些对象的引用，它们在遍历的过程中才能被找到（可以想像成有个表格，将旧地址映射至新地址）。

对于这种所谓的“复制式回收器”而言，效率会降低，这有两个原因。首先，得有两个堆，然后得在这两个分离的堆之间来回捣腾，从而得维护比实际需要多一倍的空间。某些Java虚拟机对此问题的处理方式是，按需从堆中分配几块较大的内存，复制动作发生在这些大块内存之间。

179

第二个问题在于复制。程序进入稳定状态之后，可能只会产生少量垃圾，甚至没有垃圾。

尽管如此，复制式回收器仍然会将所有内存自一处复制到另一处，这很浪费。为了避免这种情形，一些Java虚拟机会进行检查：要是没有新垃圾产生，就会转换到另一种工作模式（即“自适应”）。这种模式称为*标记-清扫*（mark-and-sweep），Sun公司早期版本的Java虚拟机使用了这种技术。对一般用途而言，“标记-清扫”方式速度相当慢，但是当你知道只会产生少量垃圾甚至不会产生垃圾时，它的速度就很快了。

“标记-清扫”所依据的思路同样是从堆栈和静态存储区出发，遍历所有的引用，进而找出所有存活的对象。每当它找到一个存活对象，就会给对象设一个标记，这个过程中不会回收任何对象。只有全部标记工作完成的时候，清理动作才会开始。在清理过程中，没有标记的对象将被释放，不会发生任何复制动作。所以剩下的堆空间是不连续的，垃圾回收器要是希望得到连续空间的话，就得重新整理剩下的对象。

“停止-复制”的意思是这种垃圾回收动作不是在后台进行的；相反，垃圾回收动作发生的同时，程序将会被暂停。在Sun 公司的文档中会发现，许多参考文献将垃圾回收视为低优先级的后台进程，但事实上垃圾回收器在Sun公司早期版本的Java虚拟机中并非以这种方式实现的。当可用内存数量较低时，Sun版本的垃圾回收器会暂停运行程序，同样，“标记-清扫”工作也必须在程序暂停的情况下才能进行。

如前文所述，在这里所讨论的Java虚拟机中，内存分配以较大的“块”为单位。如果对象较大，它会占用单独的块。严格来说，“停止-复制”要求在释放旧有对象之前，必须先把所有存活对象从旧堆复制到新堆，这将导致大量内存复制行为。有了块之后，垃圾回收器在回收的时候就可以往废弃的块里拷贝对象了。每个块都用相应的*代数*（generation count）来记录它是否还存活。通常，如果块在某处被引用，其代数会增加；垃圾回收器将对上次回收动作之后新分配的块进行整理。这对处理大量短命的临时对象很有帮助。垃圾回收器会定期进行完整的清理动作——大型对象仍然不会被复制（只是其代数会增加），内含小型对象的那些块则被复制并整理。Java虚拟机会进行监视，如果所有对象都很稳定，垃圾回收器的效率降低的话，就切换到“标记-清扫”方式；同样，Java虚拟机会跟踪“标记-清扫”的效果，要是堆空间出现很多碎片，就会切换回“停止-复制”方式。这就是“自适应”技术，你可以给它个罗嗦的称呼：“自适应的、分代的、停止-复制、标记-清扫”式垃圾回收器。 180

Java虚拟机中有许多附加技术用以提升速度。尤其是与加载器操作有关的，被称为“即时”（Just-In-Time，JIT）编译器的技术。这种技术可以把程序全部或部分翻译成本地机器码（这本来是Java虚拟机的工作），程序运行速度因此得以提升。当需要装载某个类（通常是在为该类创建第一个对象）时，编译器会先找到其 **.class** 文件，然后将该类的字节码装入内存。此时，有两种方案可供选择。一种是就让即时编译器编译所有代码。但这种做法有两个缺陷：这种加载动作散落在整个程序生命周期内，累加起来要花更多时间；并且会增加可执行代码的长度（字节码要比即时编译器展开后的本地机器码小很多），这将导致页面调度，从而降低程序速度。另一种做法称为*惰性评估*（lazy evaluation），意思是即时编译器只在必要的时候才编译代码。这样，从不会被执行的代码也许就压根不会被JIT所编译。新版JDK中的Java HotSpot技术就采用了类似方法，代码每次被执行的时候都会做一些优化，所以执行的次数越多，它的速度就越快。

5.6 成员初始化

Java尽力保证：所有变量在使用前都能得到恰当的初始化。对于方法的局部变量，Java以编译时错误的形式来贯彻这种保证。所以如果写成：

```
void f() {
  int i;
  i++; // Error -- i not initialized
}
```

就会得到一条出错消息，告诉你i可能尚未初始化。当然，编译器也可以为i赋一个默认值，但是未初始化的局部变量更有可能是程序员的疏忽，所以采用默认值反而会掩盖这种失误。因此强制程序员提供一个初始值，往往能够帮助找出程序里的缺陷。

181

要是类的数据成员（即字段）是基本类型，情况就会变得有些不同。正如在“一切都是对象”一章中所看到的，类的每个基本类型数据成员保证都会有一个初始值。下面的程序可以验证这类情况，并显示它们的值：

```
//: initialization/InitialValues.java
// Shows default initial values.
import static net.mindview.util.Print.*;

public class InitialValues {
  boolean t;
  char c;
  byte b;
  short s;
  int i;
  long l;
  float f;
  double d;
  InitialValues reference;
  void printInitialValues() {
    print("Data type      Initial value");
    print("boolean        " + t);
    print("char           [" + c + "]");
    print("byte           " + b);
    print("short          " + s);
    print("int            " + i);
    print("long           " + l);
    print("float          " + f);
    print("double         " + d);
    print("reference      " + reference);
  }
  public static void main(String[] args) {
    InitialValues iv = new InitialValues();
    iv.printInitialValues();
    /* You could also say:
    new InitialValues().printInitialValues();
    */
  }
} /* Output:
Data type      Initial value
boolean        false
char           [ ]
byte           0
short          0
int            0
long           0
float          0.0
double         0.0
reference      null
*///:~
```

182

可见尽管数据成员的初值没有给出，但它们确实有初值（**char**值为0，所以显示为空白）。这样至少不会冒“未初始化变量”的风险了。

在类里定义一个对象引用时，如果不将其初始化，此引用就会获得一个特殊值**null**。

5.6.1 指定初始化

如果想为某个变量赋初值，该怎么做呢？有一种很直接的办法，就是在定义类成员变量的地方为其赋值（注意在C++里不能这样做，尽管C++的新手们总想这样做）。以下代码片段修改了**InitialValues**类成员变量的定义，直接提供了初值。

```
//: initialization/InitialValues2.java
// Providing explicit initial values.

public class InitialValues2 {
  boolean bool = true;
  char ch = 'x';
  byte b = 47;
  short s = 0xff;
  int i = 999;
  long lng = 1;
  float f = 3.14f;
  double d = 3.14159;
} ///:~
```

也可以用同样的方法初始化非基本类型的对象。如果**Depth**是一个类，那么可以像下面这样创建一个对象并初始化它：

```
//: initialization/Measurement.java
class Depth {}

public class Measurement {
  Depth d = new Depth();
  // ...
} ///:~
```

183

如果没有为**d**指定初始值就尝试使用它，就会出现运行时错误，告诉你产生了一个异常（这在第12章中详述）。

甚至可以通过调用某个方法来提供初值：

```
//: initialization/MethodInit.java
public class MethodInit {
  int i = f();
  int f() { return 11; }
} ///:~
```

这个方法也可以带有参数，但这些参数必须是已经被初始化了的。因此，可以这样写：

```
//: initialization/MethodInit2.java
public class MethodInit2 {
  int i = f();
  int j = g(i);
  int f() { return 11; }
  int g(int n) { return n * 10; }
} ///:~
```

但像下面这样写就不对了：

```
//: initialization/MethodInit3.java
public class MethodInit3 {
  //! int j = g(i); // Illegal forward reference
  int i = f();
  int f() { return 11; }
  int g(int n) { return n * 10; }
} ///:~
```

显然，上述程序的正确性取决于初始化的顺序，而与其编译方式无关。所以，编译器恰当地对“向前引用”发出了警告。

这种初始化方法既简单又直观。但有个限制：类**InitialValues**的每个对象都会具有相同的初

184 值。有时，这正是所希望的，但有时却需要更大的灵活性。

5.7 构造器初始化

可以用构造器来进行初始化。在运行时刻，可以调用方法或执行某些动作来确定初值，这为编程带来了更大的灵活性。但要牢记：无法阻止自动初始化的进行，它将在构造器被调用之前发生。因此，假如使用下述代码：

```
//: initialization/Counter.java
public class Counter {
  int i;
  Counter() { i = 7; }
  // ...
} ///:~
```

那么i首先会被置0，然后变成7。对于所有基本类型和对象引用，包括在定义时已经指定初值的变量，这种情况都是成立的；因此，编译器不会强制你一定要在构造器的某个地方或在使用它们之前对元素进行初始化——因为初始化早已得到了保证。

5.7.1 初始化顺序

在类的内部，变量定义的先后顺序决定了初始化的顺序。即使变量定义散布于方法定义之间，它们仍旧会在任何方法（包括构造器）被调用之前得到初始化。例如：

```
//: initialization/OrderOfInitialization.java
// Demonstrates initialization order.
import static net.mindview.util.Print.*;

// When the constructor is called to create a
// Window object, you'll see a message:
class Window {
  Window(int marker) { print("Window(" + marker + ")"); }
}

class House {
  Window w1 = new Window(1); // Before constructor
  House() {
    // Show that we're in the constructor:
    print("House()");
    w3 = new Window(33); // Reinitialize w3
  }
  Window w2 = new Window(2); // After constructor
  void f() { print("f()"); }
  Window w3 = new Window(3); // At end
}

public class OrderOfInitialization {
  public static void main(String[] args) {
    House h = new House();
    h.f(); // Shows that construction is done
  }
} /* Output:
Window(1)
Window(2)
Window(3)
House()
Window(33)
f()
*///:~
```

185

在House类中，故意把几个Window对象的定义散布到各处，以证明它们全都会在调用构造器或其他方法之前得到初始化。此外，w3在构造器内再次被初始化。

由输出可见，**w3**这个引用会被初始化两次：一次在调用构造器前，一次在调用期间（第一次引用的对象将被丢弃，并作为垃圾回收）。试想，如果定义了一个重载构造器，它没有初始化**w3**；同时在**w3**的定义里也没有指定默认值，那会产生什么后果呢？所以尽管这种方法似乎效率不高，但它的确能使初始化得到保证。

5.7.2 静态数据的初始化

无论创建多少个对象，静态数据都只占用一份存储区域。**static**关键字不能应用于局部变量，因此它只能作用于域。如果一个域是静态的基本类型域，且也没有对它进行初始化，那么它就会获得基本类型的标准初值；如果它是一个对象引用，那么它的默认初始化值就是**null**。 186

如果想在定义处进行初始化，采取的方法与非静态数据没什么不同。

要想了解静态存储区域是何时初始化的，就请看下面这个例子：

```
//: initialization/StaticInitialization.java
// Specifying initial values in a class definition.
import static net.mindview.util.Print.*;

class Bowl {
  Bowl(int marker) {
    print("Bowl(" + marker + ")");
  }
  void f1(int marker) {
    print("f1(" + marker + ")");
  }
}

class Table {
  static Bowl bowl1 = new Bowl(1);
  Table() {
    print("Table()");
    bowl2.f1(1);
  }
  void f2(int marker) {
    print("f2(" + marker + ")");
  }
  static Bowl bowl2 = new Bowl(2);
}

class Cupboard {
  Bowl bowl3 = new Bowl(3);
  static Bowl bowl4 = new Bowl(4);
  Cupboard() {
    print("Cupboard()");
    bowl4.f1(2);
  }
  void f3(int marker) {
    print("f3(" + marker + ")");
  }
  static Bowl bowl5 = new Bowl(5);
}

public class StaticInitialization {
  public static void main(String[] args) {
    print("Creating new Cupboard() in main");
    new Cupboard();
    print("Creating new Cupboard() in main");
    new Cupboard();
    table.f2(1);
    cupboard.f3(1);
  }
  static Table table = new Table();
  static Cupboard cupboard = new Cupboard();
} /* Output:
```

```
Bowl(1)
Bowl(2)
Table()
f1(1)
Bowl(4)
Bowl(5)
Bowl(3)
Cupboard()
f1(2)
Creating new Cupboard() in main
Bowl(3)
Cupboard()
f1(2)
Creating new Cupboard() in main
Bowl(3)
Cupboard()
f1(2)
f2(1)
f3(1)
*///:~
```

Bowl类使得看到类的创建，而**Table**类和**Cupboard**类在它们的类定义中加入了**Bowl**类型的静态数据成员。注意，在静态数据成员定义之前，**Cupboard**类先定义了一个**Bowl**类型的非静态数据成员 **b3**。

由输出可见，静态初始化只有在必要时刻才会进行。如果不创建**Table**对象，也不引用**Table.b1**或**Table.b2**，那么静态的**Bowl b1**和**b2**永远都不会被创建。只有在第一个Table对象被创建（或者第一次访问静态数据）的时候，它们才会被初始化。此后，静态对象不会再次被初始化。

188

初始化的顺序是先静态对象（如果它们尚未因前面的对象创建过程而被初始化），而后是“非静态”对象。从输出结果中可以观察到这一点。要执行**main()**（静态方法），必须加载**StaticInitialization**类，然后其静态域**table**和**cupboard**被初始化，这将导致它们对应的类也被加载，并且由于它们也都包含静态的**Bowl**对象，因此**Bowl**随后也被加载。这样，在这个特殊的程序中的所有类在**main()**开始之前就都被加载了。实际情况通常并非如此，因为在典型的程序中，不会像在本例中所做的那样，将所有的事物都通过**static**联系起来。

总结一下对象的创建过程，假设有个名为**Dog**的类：

1. 即使没有显式地使用**static**关键字，构造器实际上也是静态方法。因此，当首次创建类型为**Dog**的对象时（构造器可以看成静态方法），或者**Dog**类的静态方法/静态域首次被访问时，Java解释器必须查找类路径，以定位**Dog.class**文件。

2. 然后载入**Dog.class**（后面会学到，这将创建一个**Class**对象），有关静态初始化的所有动作都会执行。因此，静态初始化只在**Class**对象首次加载的时候进行一次。

3. 当用**new Dog()**创建对象的时候，首先将在堆上为**Dog**对象分配足够的存储空间。

4. 这块存储空间会被清零，这就自动地将**Dog**对象中的所有基本类型数据都设置成了默认值（对数字来说就是0，对布尔型和字符型也相同），而引用则被设置成了**null**。

5. 执行所有出现于字段定义处的初始化动作。

6. 执行构造器。正如将在第7章所看到的，这可能会牵涉到很多动作，尤其是涉及继承的时候。

189

5.7.3 显式的静态初始化

Java允许将多个静态初始化动作组织成一个特殊的“静态子句”（有时也叫做“静态块”）。就像下面这样：

```
//: initialization/Spoon.java
public class Spoon {
  static int i;
  static {
    i = 47;
  }
} ///:~
```

尽管上面的代码看起来像个方法，但它实际只是一段跟在**static**关键字后面的代码。与其他静态初始化动作一样，这段代码仅执行一次：当首次生成这个类的一个对象时，或者首次访问属于那个类的静态数据成员时（即便从未生成过那个类的对象）。例如：

```
//: initialization/ExplicitStatic.java
// Explicit static initialization with the "static" clause.
import static net.mindview.util.Print.*;

class Cup {
  Cup(int marker) {
    print("Cup(" + marker + ")");
  }
  void f(int marker) {
    print("f(" + marker + ")");
  }
}

class Cups {
  static Cup cup1;
  static Cup cup2;
  static {
    cup1 = new Cup(1);
    cup2 = new Cup(2);
  }
  Cups() {
    print("Cups()");
  }
}
public class ExplicitStatic {
  public static void main(String[] args) {
    print("Inside main()");
    Cups.cup1.f(99);  // (1)
  }
  // static Cups cups1 = new Cups();  // (2)
  // static Cups cups2 = new Cups();  // (2)
} /* Output:
Inside main()
Cup(1)
Cup(2)
f(99)
*///:~
```

190

无论是通过标为**(1)**的那行代码访问静态的**cup1**对象，还是把标为**(1)**的行注释掉，让它去运行标为**(2)**的那行代码（即解除标为**(2)**的行的注释），**Cups**的静态初始化动作都会得到执行。如果把标为**(1)**和**(2)**的行同时注释掉，**Cups**的静态初始化动作就不会进行，就像在输出中看到的那样。此外，激活一行还是两行标为**(2)**的代码（即解除注释）都无关紧要，静态初始化动作只进行一次。

练习13：(1) 验证前面段落中的语句。

练习14：(1) 编写一个类，拥有两个静态字符串域，其中一个在定义处初始化，另一个在静态块中初始化。现在，加入一个静态方法用以打印出两个字段值。请证明它们都会在被使用之前完成初始化动作。

5.7.4 非静态实例初始化

Java中也有被称为实例初始化的类似语法，用来初始化每一个对象的非静态变量。例如：

```
//: initialization/Mugs.java
// Java "Instance Initialization."
import static net.mindview.util.Print.*;

class Mug {
  Mug(int marker) {
    print("Mug(" + marker + ")");
  }
  void f(int marker) {
    print("f(" + marker + ")");
  }
}
public class Mugs {
  Mug mug1;
  Mug mug2;
  {
    mug1 = new Mug(1);
    mug2 = new Mug(2);
    print("mug1 & mug2 initialized");
  }
  Mugs() {
    print("Mugs()");
  }
  Mugs(int i) {
    print("Mugs(int)");
  }
  public static void main(String[] args) {
    print("Inside main()");
    new Mugs();
    print("new Mugs() completed");
    new Mugs(1);
    print("new Mugs(1) completed");
  }
} /* Output:
Inside main()
Mug(1)
Mug(2)
mug1 & mug2 initialized
Mugs()
new Mugs() completed
Mug(1)
Mug(2)
mug1 & mug2 initialized
Mugs(int)
new Mugs(1) completed
*///:~
```

191

你可以看到实例初始化子句：

```
{
  mug1 = new Mug(1);
  mug2 = new Mug(2);
  print("mug1 & mug2 initialized");
}
```

192

看起来它与静态初始化子句一模一样，只不过少了static关键字。这种语法对于支持“匿名内部类”（参见第10章）的初始化是必须的，但是它也使得你可以保证无论调用了哪个显式构造器，某些操作都会发生。从输出中可以看到实例初始化子句是在两个构造器之前执行的。

练习15：(1) 编写一个含有字符串域的类，并采用实例初始化方式进行初始化。

5.8 数组初始化

数组只是相同类型的、用一个标识符名称封装到一起的一个对象序列或基本类型数据序列。数组是通过方括号下标操作符[ ]来定义和使用的。要定义一个数组，只需在类型名后加上一对空方括号即可：

```
int[] a1;
```

方括号也可以置于标识符后面：

```
int a1[];
```

两种格式的含义是一样的，后一种格式符合C和C++程序员的习惯。不过，前一种格式或许更合理，毕竟它表明类型是“一个**int**型数组”。本书将采用这种格式。

编译器不允许指定数组的大小。这就又把我们带回到有关“引用”的问题上。现在拥有的只是对数组的一个引用（你已经为该引用分配了足够的存储空间），而且也没给数组对象本身分配任何空间。为了给数组创建相应的存储空间，必须写初始化表达式。对于数组，初始化动作可以出现在代码的任何地方，但也可以使用一种特殊的初始化表达式，它必须在创建数组的地方出现。这种特殊的初始化是由一对花括号括起来的值组成的。在这种情况下，存储空间的分配（等价于使用**new**）将由编译器负责。例如：

```
int[] a1 = { 1, 2, 3, 4, 5 };
```

那么，为什么还要在没有数组的时候定义一个数组引用呢？ 193

```
int[] a2;
```

在Java中可以将一个数组赋值给另一个数组，所以可以这样：

```
a2 = a1;
```

其实真正做的只是复制了一个引用，就像下面演示的那样：

```
//: initialization/ArraysOfPrimitives.java
import static net.mindview.util.Print.*;

public class ArraysOfPrimitives {
  public static void main(String[] args) {
    int[] a1 = { 1, 2, 3, 4, 5 };
    int[] a2;
    a2 = a1;
    for(int i = 0; i < a2.length; i++)
      a2[i] = a2[i] + 1;
    for(int i = 0; i < a1.length; i++)
      print("a1[" + i + "] = " + a1[i]);
  }
} /* Output:
a1[0] = 2
a1[1] = 3
a1[2] = 4
a1[3] = 5
a1[4] = 6
*///:~
```

可以看到代码中给出了**a1**的初始值，但**a2**却没有；在本例中，**a2**是在后面被赋给另一个数组的。由于**a2**和**a1**是相同数组的别名，因此通过**a2**所做的修改在**a1**中可以看到。

所有数组（无论它们的元素是对象还是基本类型）都有一个固有成员，可以通过它获知数组内包含了多少个元素，但不能对其修改。这个成员就是**length**。与C和C++类似，Java数组计数也是从第0个元素开始，所以能使用的最大下标数是**length-1**。要是超出这个边界，C和C++会“默默”地接受，并允许你访问所有内存，许多声名狼藉的程序错误由此而生。Java则能保护你免受这一问题的困扰，一旦访问下标过界，就会出现运行时错误（即异常）[⊖]。 194

⊖ 当然，每次访问数组的时候都要检查边界的做法在时间和代码上都是需要开销的，但是无法禁用这个功能。这意味着如果数组访问发生在一些关键节点上，它们有可能会成为导致程序效率低下的原因之一。但是基于“因特网的安全以及提高程序员生产力”的理由，Java的设计者认为这种权衡是值得的。尽管你可能会受到诱惑，去编写你认为可以使得数组访问效率提高的代码，但是这一切都是在浪费时间，因为自动的编译期错误和运行时优化都可以提高数组访问的速度。

如果在编写程序时，并不能确定在数组里需要多少个元素，那么该怎么办呢？可以直接用**new**在数组里创建元素。尽管创建的是基本类型数组，**new**仍然可以工作（不能用**new**创建单个的基本类型数据）。

```
//: initialization/ArrayNew.java
// Creating arrays with new.
import java.util.*;
import static net.mindview.util.Print.*;

public class ArrayNew {
  public static void main(String[] args) {
    int[] a;
    Random rand = new Random(47);
    a = new int[rand.nextInt(20)];
    print("length of a = " + a.length);
    print(Arrays.toString(a));
  }
} /* Output:
length of a = 18
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
*///:~
```

数组的大小是通过**Random.nextInt()**方法随机决定的，这个方法会返回0到输入参数之间的一个值。这表明数组的创建确实是在运行时刻进行的。此外，程序输出表明：数组元素中的基本数据类型值会自动初始化成空值（对于数字和字符，就是0；对于布尔型，是**false**）。

195 **Arrays.toString()**方法属于**java.util**标准类库，它将产生一维数组的可打印版本。

当然，在本例中，数组也可以在定义的同时进行初始化：

```
int[] a = new int[rand.nextInt(20)];
```

如果可能的话，应该尽量这么做。

如果你创建了一个非基本类型的数组，那么你就创建了一个引用数组。以整型的包装器类**Integer**为例，它是一个类而不是基本类型：

```
//: initialization/ArrayClassObj.java
// Creating an array of nonprimitive objects.
import java.util.*;
import static net.mindview.util.Print.*;

public class ArrayClassObj {
  public static void main(String[] args) {
    Random rand = new Random(47);
    Integer[] a = new Integer[rand.nextInt(20)];
    print("length of a = " + a.length);
    for(int i = 0; i < a.length; i++)
      a[i] = rand.nextInt(500); // Autoboxing
    print(Arrays.toString(a));
  }
} /* Output: (Sample)
length of a = 18
[55, 193, 361, 461, 429, 368, 200, 22, 207, 288, 128, 51,
89, 309, 278, 498, 361, 20]
*///:~
```

这里，即便使用**new**创建数组之后：

```
Integer[] a = new Integer[rand.nextInt(20)];
```

它还只是一个引用数组，并且直到通过创建新的**Integer**对象（在本例中是通过自动包装机制创建的），并把对象赋值给引用，初始化进程才算结束：

```
a[i] = rand.nextInt(500);
```

如果忘记了创建对象，并且试图使用数组中的空引用，就会在运行时产生异常。

也可以用花括号括起来的列表来初始化对象数组。有两种形式： 196

```
//: initialization/ArrayInit.java
// Array initialization.
import java.util.*;

public class ArrayInit {
  public static void main(String[] args) {
    Integer[] a = {
      new Integer(1),
      new Integer(2),
      3, // Autoboxing
    };
    Integer[] b = new Integer[]{
      new Integer(1),
      new Integer(2),
      3, // Autoboxing
    };
    System.out.println(Arrays.toString(a));
    System.out.println(Arrays.toString(b));
  }
} /* Output:
[1, 2, 3]
[1, 2, 3]
*///:~
```

在这两种形式中，初始化列表的最后一个逗号都是可选的（这一特性使维护长列表变得更容易）。

尽管第一种形式很有用，但是它也更加受限，因为它只能用于数组被定义之处。你可以在任何地方使用第二种和第三种形式，甚至是在方法调用的内部。例如，你可以创建一个**String**对象数组，将其传递给另一个**main()**方法，以提供参数，用来替换传递给该**main()**方法的命令行参数。

```
//: initialization/DynamicArray.java
// Array initialization.

public class DynamicArray {
  public static void main(String[] args) {
    Other.main(new String[]{ "fiddle", "de", "dum" });
  }
}

class Other {
  public static void main(String[] args) {
    for(String s : args)
      System.out.print(s + " ");
  }
} /* Output:
fiddle de dum
*///:~
```

197

为**Other.main()**的参数而创建的数组是在方法调用处创建的，因此你甚至可以在调用时提供可替换的参数。

练习16：(1) 创建一个**String**对象数据，并为每一个元素都赋值一个**String**。用for循环来打印该数组。

练习17：(2) 创建一个类，它有一个接受一个**String**参数的构造器。在构造阶段，打印该参数。创建一个该类的对象引用数组，但是不实际去创建对象赋值给该数组。当运行程序时，请注意来自对该构造器的调用中的初始化消息是否打印了出来。

练习18：(1) 通过创建对象赋值给引用数组，从而完成前一个练习。

5.8.1 可变参数列表

第二种形式提供了一种方便的语法来创建对象并调用方法，以获得与C的可变参数列表（C通常把它简称为varargs）一样的效果。这可以应用于参数个数或类型未知的场合。由于所有的类都直接或间接继承于**Object**类（随着本书的进展，读者会对此有更深入的认识），所以可以创建以**Object**数组为参数的方法，并像下面这样调用：

```
//: initialization/VarArgs.java
// Using array syntax to create variable argument lists.

class A {}

public class VarArgs {
  static void printArray(Object[] args) {
    for(Object obj : args)
      System.out.print(obj + " ");
    System.out.println();
  }
  public static void main(String[] args) {
    printArray(new Object[]{
      new Integer(47), new Float(3.14), new Double(11.11)
    });
    printArray(new Object[]{"one", "two", "three" });
    printArray(new Object[]{new A(), new A(), new A()});
  }
} /* Output: (Sample)
47 3.14 11.11
one two three
A@1a46e30 A@3e25a5 A@19821f
*///:~
```

198

可以看到**print()**方法使用**Object**数组作为参数，然后使用foreach语法遍历数组，打印每个对象。标准Java库中的类能输出有意义的内容，但这里建立的类的对象，打印出的内容只是类的名称以及后面紧跟着的一个@符号以及多个十六进制数字。于是，默认行为（如果没有定义**toString()**方法的话，后面会讲这个方法的）就是打印类的名字和对象的地址。

你可能看到过像上面这样编写的Java SE5之前的代码，它们可以产生可变的参数列表。然而，在Java SE5中，这种盼望已久的特性终于添加了进来，因此你现在可以使用它们来定义可变参数列表了，就像在**printArray()**中看到的那样：

```
//: initialization/NewVarArgs.java
// Using array syntax to create variable argument lists.

public class NewVarArgs {
  static void printArray(Object... args) {
    for(Object obj : args)
      System.out.print(obj + " ");
    System.out.println();
  }
  public static void main(String[] args) {
    // Can take individual elements:
    printArray(new Integer(47), new Float(3.14),
      new Double(11.11));
    printArray(47, 3.14F, 11.11);
    printArray("one", "two", "three");
    printArray(new A(), new A(), new A());
    // Or an array:
    printArray((Object[])new Integer[]{ 1, 2, 3, 4 });
    printArray(); // Empty list is OK
  }
} /* Output: (75% match)
47 3.14 11.11
47 3.14 11.11
```

199

```
one two three
A@1bab50a A@c3c749 A@150bd4d
1 2 3 4
*///:~
```

有了可变参数，就再也不用显式地编写数组语法了，当你指定参数时，编译器实际上会为你去填充数组。你获取的仍旧是一个数组，这就是为什么**print()**可以使用foreach来迭代该数组的原因。但是，这不仅仅只是从元素列表到数组的自动转换，请注意程序中倒数第二行，一个**Integer**数组（通过使用自动包装而创建的）被转型为一个**Object**数组（以便移除编译器警告信息），并且传递给了**printArray()**。很明显，编译器会发现它已经是一个数组了，所以不会在其上执行任何转换。因此，如果你有一组事物，可以把它们当作列表传递，而如果你已经有了一个数组，该方法可以把它们当作可变参数列表来接受。

该程序的最后一行表明将0个参数传递给可变参数列表是可行的，当具有可选的尾随参数时，这一特性就会很有用：

```
//: initialization/OptionalTrailingArguments.java

public class OptionalTrailingArguments {
  static void f(int required, String... trailing) {
    System.out.print("required: " + required + " ");
    for(String s : trailing)
      System.out.print(s + " ");
    System.out.println();
  }
  public static void main(String[] args) {
    f(1, "one");
    f(2, "two", "three");
    f(0);
  }
} /* Output:
required: 1 one
required: 2 two three
required: 0
*///:~
```

200

这个程序还展示了你可以如何使用具有**Object**之外类型的可变参数列表。这里所有的可变参数都必须是**String**对象。在可变参数列表中可以使用任何类型的参数，包括基本类型。下面的例子也展示了可变参数列表变为数组的情形，并且如果在该列表中没有任何元素，那么转变成的数据的尺寸为0：

```
//: initialization/VarargType.java

public class VarargType {
  static void f(Character... args) {
    System.out.print(args.getClass());
    System.out.println(" length " + args.length);
  }
  static void g(int... args) {
    System.out.print(args.getClass());
    System.out.println(" length " + args.length);
  }
  public static void main(String[] args) {
    f('a');
    f();
    g(1);
    g();
    System.out.println("int[]: " + new int[0].getClass());
  }
} /* Output:
class [Ljava.lang.Character; length 1
class [Ljava.lang.Character; length 0
```

```
class [I length 1
class [I length 0
int[]: class [I
*///:~
```

getClass()方法属于**Object**的一部分，我们将在第14章中做全面介绍。它将产生对象的类，并且在打印该类时，可以看到表示该类类型的编码字符串。前导的“**[**”表示这是一个后面紧随的类型的数组，而紧随的“**I**”表示基本类型**int**。为了进行双重检查，我在最后一行创建了一个**int**数组，并打印了其类型。这样也就验证了使用可变参数列表不依赖于自动包装机制，而实际上使用的是基本类型。

然而，可变参数列表与自动包装机制可以和谐共处，例如：

201

```
//: initialization/AutoboxingVarargs.java
public class AutoboxingVarargs {
  public static void f(Integer... args) {
    for(Integer i : args)
      System.out.print(i + " ");
    System.out.println();
  }
  public static void main(String[] args) {
    f(new Integer(1), new Integer(2));
    f(4, 5, 6, 7, 8, 9);
    f(10, new Integer(11), 12);
  }
} /* Output:
1 2
4 5 6 7 8 9
10 11 12
*///:~
```

请注意，你可以在单一的参数列表中将类型混合在一起，而自动包装机制将有选择地将**int**参数提升为**Integer**。

可变参数列表使得重载过程变得复杂了，尽管乍一看会显得足够安全：

```
//: initialization/OverloadingVarargs.java

public class OverloadingVarargs {
  static void f(Character... args) {
    System.out.print("first");
    for(Character c : args)
      System.out.print(" " + c);
    System.out.println();
  }
  static void f(Integer... args) {
    System.out.print("second");
    for(Integer i : args)
      System.out.print(" " + i);
    System.out.println();
  }
  static void f(Long... args) {
    System.out.println("third");
  }
  public static void main(String[] args) {
    f('a', 'b', 'c');
    f(1);
    f(2, 1);
    f(0);
    f(0L);
    //! f(); // Won't compile -- ambiguous
  }
} /* Output:
first a b c
second 1
```

202

```
second 2 1
second 0
third
*///:~
```

在每一种情况中，编译器都会使用自动包装机制来匹配重载的方法，然后调用最明确匹配的方法。

但是在不使用参数调用f()时，编译器就无法知道应该调用哪一个方法了。尽管这个错误可以弄清楚，但是它可能会使客户端程序员大感意外。

你可能会通过在某个方法中增加一个非可变参数来解决该问题：

```
//: initialization/OverloadingVarargs2.java
// {CompileTimeError} (Won't compile)

public class OverloadingVarargs2 {
  static void f(float i, Character... args) {
    System.out.println("first");
  }
  static void f(Character... args) {
    System.out.print("second");
  }
  public static void main(String[] args) {
    f(1, 'a');
    f('a', 'b');
  }
} ///:~
```

{CompileTimeError}注释标签把该文件排除在了本书的Ant构建之外。如果你手动编译它，就会得到下面的错误消息：

reference to f is ambiguous, both method f(float,java.lang.Character...)
in OverloadingVarargs2 and method f(java.lang.Character...) in
OverloadingVarargs2 match

203

如果你给这两个方法都添加一个非可变参数，就可以解决问题了：

```
//: initialization/OverloadingVarargs3.java

public class OverloadingVarargs3 {
  static void f(float i, Character... args) {
    System.out.println("first");
  }
  static void f(char c, Character... args) {
    System.out.println("second");
  }
  public static void main(String[] args) {
    f(1, 'a');
    f('a', 'b');
  }
} /* Output:
first
second
*///:~
```

你应该总是只在重载方法的一个版本上使用可变参数列表，或者压根就不是用它。

练习19：(2) 写一个类，它接受一个可变参数的**String**数组。验证你可以向该方法传递一个用逗号分隔的**String**列表，或是一个**String[]**。

练习20：(1) 创建一个使用可变参数列表而不是普通的**main()**语法的**main()**。打印所产生的**args**数组的所有元素，并用各种不同数量的命令行参数来测试它。

5.9 枚举类型

在Java SE5中添加了一个看似很小的特性，即**enum**关键字，它使得我们在需要群组并使用

枚举类型集时，可以很方便地处理。在此之前，你需要创建一个整型常量集，但是这些枚举值并不会必然地将其自身的取值限制在这个常量集的范围之内，因此它们显得更有风险，且更难以使用。枚举类型属于非常普遍的需求，C、C++和其他许多语言都已经拥有它了。在Java SE5之前，Java程序员在需要使用枚举类型时，必须了解很多细节并需要格外仔细，以正确地产生**enum**的效果。现在Java也有了**enum**，并且它的功能比C/C++中的枚举类型要完备得多。下面是一个简单的例子：

204

```
//: initialization/Spiciness.java
public enum Spiciness {
  NOT, MILD, MEDIUM, HOT, FLAMING
} ///:~
```

这里创建了一个名为**Spiciness**的枚举类型，它具有5个具名值。由于枚举类型的实例是常量，因此按照命名惯例它们都用大写字母表示（如果在一个名字中有多个单词，用下划线将它们隔开）。

为了使用**enum**，需要创建一个该类型的引用，并将其赋值给某个实例：

```
//: initialization/SimpleEnumUse.java
public class SimpleEnumUse {
  public static void main(String[] args) {
    Spiciness howHot = Spiciness.MEDIUM;
    System.out.println(howHot);
  }
} /* Output:
MEDIUM
*///:~
```

在你创建**enum**时，编译器会自动添加一些有用的特性。例如，它会创建**toString()**方法，以便你可以很方便地显示某个**enum**实例的名字，这正是上面的打印语句如何产生其输出的答案。编译器还会创建**ordinal()**方法，用来表示某个特定**enum**常量的声明顺序，以及**static values()**方法，用来按照**enum**常量的声明顺序，产生由这些常量值构成的数组：

```
//: initialization/EnumOrder.java
public class EnumOrder {
  public static void main(String[] args) {
    for(Spiciness s : Spiciness.values())
      System.out.println(s + ", ordinal " + s.ordinal());
  }
} /* Output:
NOT, ordinal 0
MILD, ordinal 1
MEDIUM, ordinal 2
HOT, ordinal 3
FLAMING, ordinal 4
*///:~
```

205

尽管**enum**看起来像是一种新的数据类型，但是这个关键字只是为**enum**生成对应的类时，产生了某些编译器行为，因此在很大程度上，你可以将**enum**当作其他任何类来处理。事实上，**enum**确实是类，并且具有自己的方法。

enum有一个特别实用的特性，即它可以在**switch**语句内使用：

```
//: initialization/Burrito.java

public class Burrito {
  Spiciness degree;
  public Burrito(Spiciness degree) { this.degree = degree;}
  public void describe() {
    System.out.print("This burrito is ");
    switch(degree) {
```

```
        case NOT:     System.out.println("not spicy at all.");
                      break;
        case MILD:
        case MEDIUM: System.out.println("a little hot.");
                      break;
        case HOT:
        case FLAMING:
        default:      System.out.println("maybe too hot.");
      }
    }
    public static void main(String[] args) {
      Burrito
        plain = new Burrito(Spiciness.NOT),
        greenChile = new Burrito(Spiciness.MEDIUM),
        jalapeno = new Burrito(Spiciness.HOT);
      plain.describe();
      greenChile.describe();
      jalapeno.describe();
    }
  } /* Output:
  This burrito is not spicy at all.
  This burrito is a little hot.
  This burrito is maybe too hot.
  *///:~
```

由于**switch**是要在有限的可能值集合中进行选择，因此它与**enum**正是绝佳的组合。请注意**enum**的名字是如何能够倍加清楚地表明程序意欲何为的。 206

大体上，你可以将**enum**用作另外一种创建数据类型的方式，然后直接将所得到的类型拿来使用。这正是关键所在，因此你不必过多地考虑它们。在Java SE5引进**enum**之前，你必须花费大量的精力去保证与其等价的枚举类型是安全可用的。

这些介绍对于你理解和使用基本的**enum**已经足够了，但是我们将在第19章中更加深入地探讨它们。

练习21：(1) 创建一个**enum**，它包含纸币中最小面值的6种类型。通过**values()**循环并打印每一个值及其**ordinal()**。

练习22：(2) 在前面的例子中，为**enum**写一个**switch**语句，对于每一个**case**，输出该特定货币的描述。

5.10 总结

构造器，这种精巧的初始化机制，应该给了读者很强的暗示：初始化在Java中占有至关重要的地位。C++的发明人Bjarne Stroustrup在设计C++期间，在针对C语言的生产效率所进行的最初调查中发现，大量编程错误都源于不正确的初始化。这种错误很难发现，并且不恰当的清理也会导致类似问题。构造器能保证正确的初始化和清理（没有正确的构造器调用，编译器就不允许创建对象），所以有了完全的控制，也很安全。

在C++中，“析构”相当重要，因为用new创建的对象必须明确被销毁。在Java中，垃圾回收器会自动为对象释放内存，所以在很多场合下，类似的清理方法在Java中就不太需要了（不过当要用到的时候，你就只能自己动手了）。在不需要类似析构函数的行为的时候，Java的垃圾回收器可以极大地简化编程工作，而且在处理内存的时候也更安全。有些垃圾回收器甚至能清理其他资源，比如图形和文件句柄。然而，垃圾回收器确实也增加了运行时的开销。而且Java解释器从来就很慢，所以这种开销到底造成了多大的影响也很难看出。随着时间的推移，Java在性能方面已经取得了长足的进步，但速度问题仍然是它涉足某些特定编程领域 207

的障碍。

由于要保证所有对象都被创建，构造器实际上要比这里所讨论的更复杂。特别当通过组合或继承生成新类的时候，这种保证仍然成立，并且需要一些附加的语法来提供支持。在后面的章节中，读者将学习到有关组合、继承以及它们对构造器造成的影响等方面的知识。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。

208